

Software Lifecycle Management



Fachbereich Elektrotechnik
und Technische Informatik

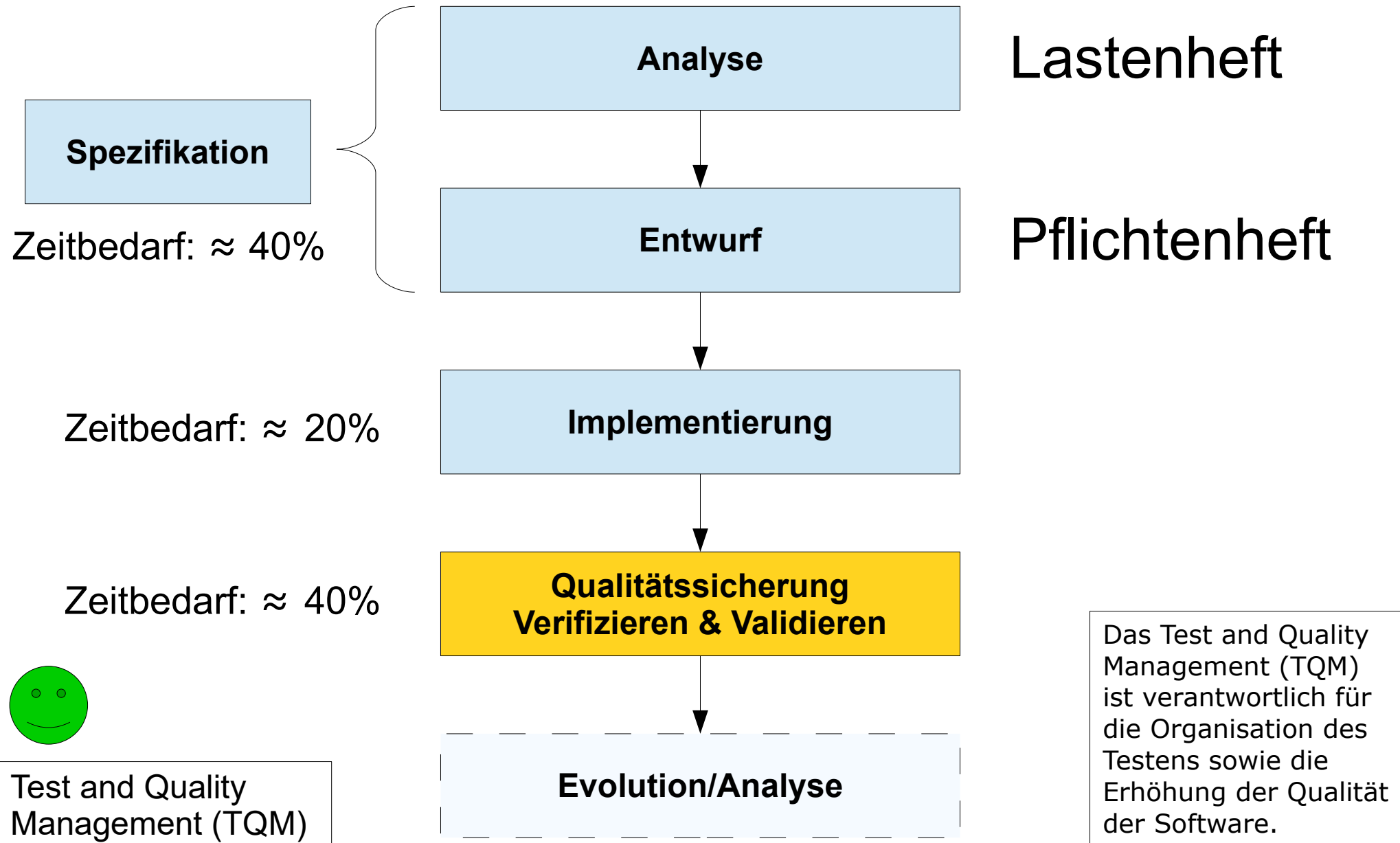
Dr. rer. nat. Nils Beckmann
nils.beckmann@th-owl.de

Testen

Themen

- Test and Quality Management (TQM)
 - Qualitätssicherung
 - Ziele, Aufgaben, Aspekte
 - Verifizierung & Validierung (V&V)
 - Statisch, Dynamisch
 - Testen
 - Phasen, Ebenen, Testfälle, Testprozess
 - Dokumente:
 - Testplan und Testbericht ("Defect Report")

Erinnerung: Softwareprozessphasen



Ziele der Qualitätssicherung

- Fehler (der Software) aufdecken
- Vertrauen erzeugen (in das Software-Produkt)
- Qualitätsniveau der Software aufzeigen und erhöhen
- Möglichen Fehlerwirkungen vorbeugen
- Nachweis, dass Anforderungen erfüllt sind

Aufgaben der Qualitätssicherung

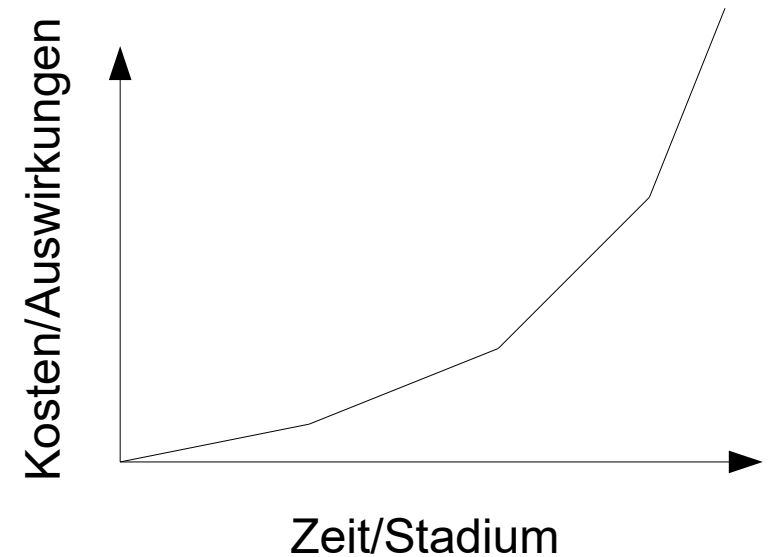
- Testplan erstellen
- Festlegen von Kriterien für das Testende
- Testfälle erstellen
- Testen = Durchführung der Testfälle
- Testergebnisse auswerten
- Rollenspezifische Testberichte erstellen
- Empfehlungen zur Freigabe geben

Aspekte der Qualitätssicherung

- Testen impliziert, dass es Fehler geben kann ("Nobody's perfect")
- Testen kann Fehler sichtbar machen
- Vollständiges Testen ist nicht möglich
- Möglichst früh mit dem Testen beginnen (wenigstens manuelle Funktions-/Klassentests)
- Testen ist individuell abhängig von der Software und vom Software-Umfeld
- Ein Trugschluss ist: "Kein Fehler gefunden bedeutet, dass das System brauchbar ist"

Auswirkungen durch Fehler

- Die Kosten und negativen Auswirkungen von Fehlern nehmen exponentiell mit dem Entwicklungsstadium zu



- Bei anspruchsvollen Systemen, z.B. Echtzeitsystemen mit komplexen nicht-funktionalen Einschränkungen und Sicherheitsanforderungen kann mehr als die Hälfte des Budgets für V&V verbraucht werden

Erinnerung: Nachverfolgbarkeit

- **Nachverfolgbarkeit** ("*traceability*"):
 - Kann ich eine Anforderung verfolgen?
 - Wie ist die Anforderung implementiert?
 - Wie ist die Anforderung getestet?
- Traceability ist wesentliches Qualitätsmerkmal für einen überwachten und strukturierten Softwareentwicklungsprozess.
 - (Digitale) Methoden und Werkzeuge des ALM unterstützen dabei.

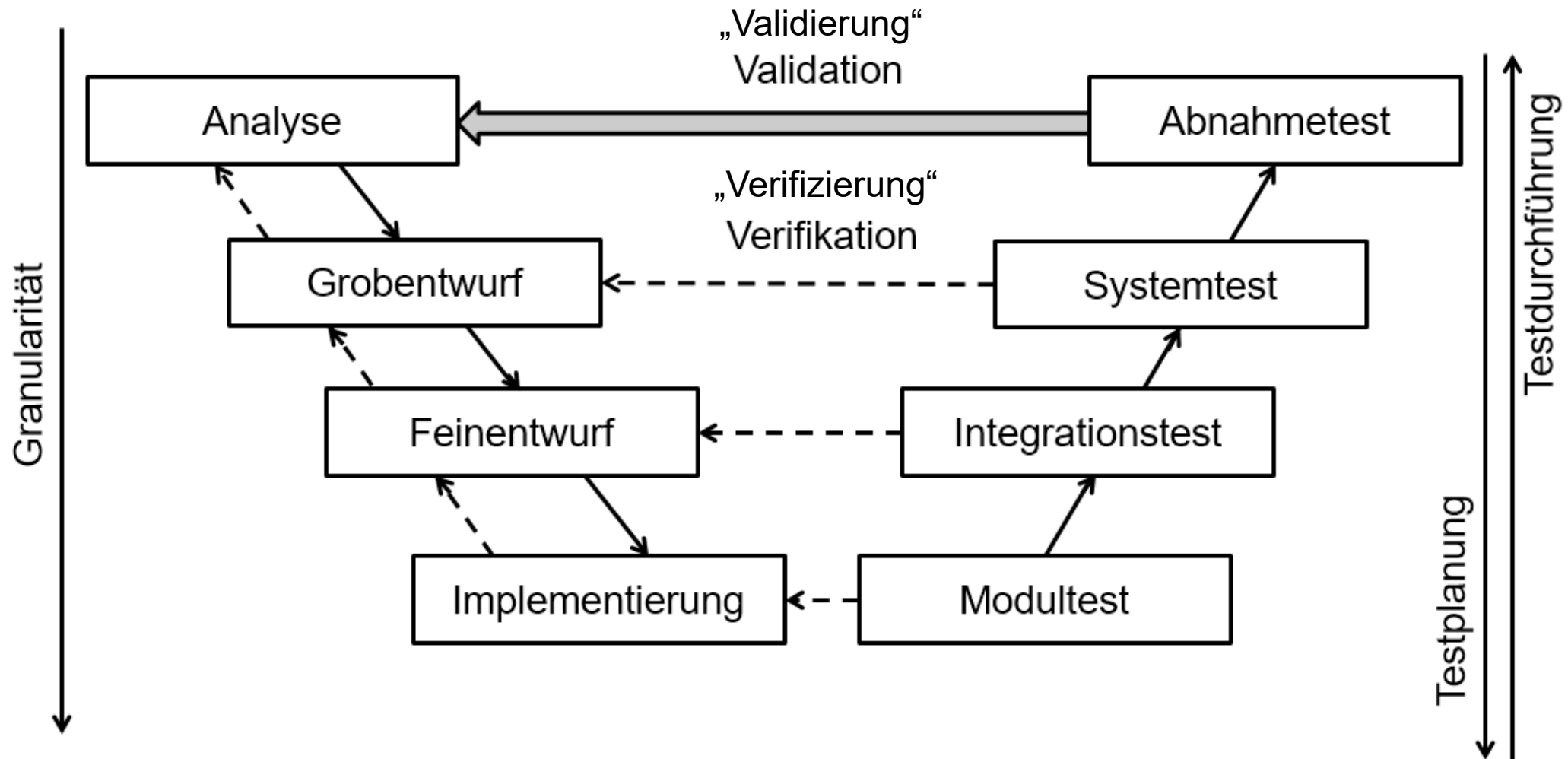
Erinnerung: Validierung und Verifizierung

Siehe Vorlesung
„107a_Klassisch“
V-Modell!

- **Validierung** ("*validation*")
 - "Was?"
 - "Entwickle ich das richtige Produkt?"
 - Anforderungen an die Nutzung
 - Fokus-Phase: Analyse
- **Verifizierung** ("*verification*")
 - "Wie?"
 - "Entwickle ich das Produkt richtig?"
 - Anforderungen an die Entwicklung
 - Fokus-Phase: Entwurf

V&V

Erinnerung: V-Modell



Definieren von Tests beginnt früh.

Statische V&V

- **Statisch:** Software wird *nicht* ausgeführt
 - z.B. Dokument (Lastenheft) lesen
 - z.B. Quelltext durchlesen und auf Fehler prüfen
 - Informelles Review: Kollege schaut auf Quelltext, Hinweise mündlich, kein Protokoll
 - Walkthrough: Durch Dokument leiten in Schritten Lernen, Verstehen, Fehler finden
 - Inspektion: Formalste statische V&V, Checkliste wird abgearbeitet, Ziel ist Fehler finden, Inspektionsprotokoll
 - Statische Code-Analyse: Zunächst durch Compiler, dann durch Zusatztools (Anomalien, z.B. nie verwendete Variablen/Funktionen, Verschachtelungstiefe, Formatierung, Namenskonventionen, Metriken, usw.), manuelles Durchsehen, etc.

Dynamische V&V

- **Dynamisch:** Software wird ausgeführt
 - Testfälle werden durchgeführt
 - Ziel ist Aufdecken von Fehlerwirkungen
- Verschiedene Ansätze:
 - Blackbox-Verfahren: Testfälle aus Spezifikation
 - Whitebox-Verfahren: Testfälle aus Programmstruktur
 - Erfahrungsbasierte Ansätze: Testfälle werden basierend auf Mitarbeitererfahrung erstellt
 - Unverzichtbar bei erfahrenen Mitarbeitern!

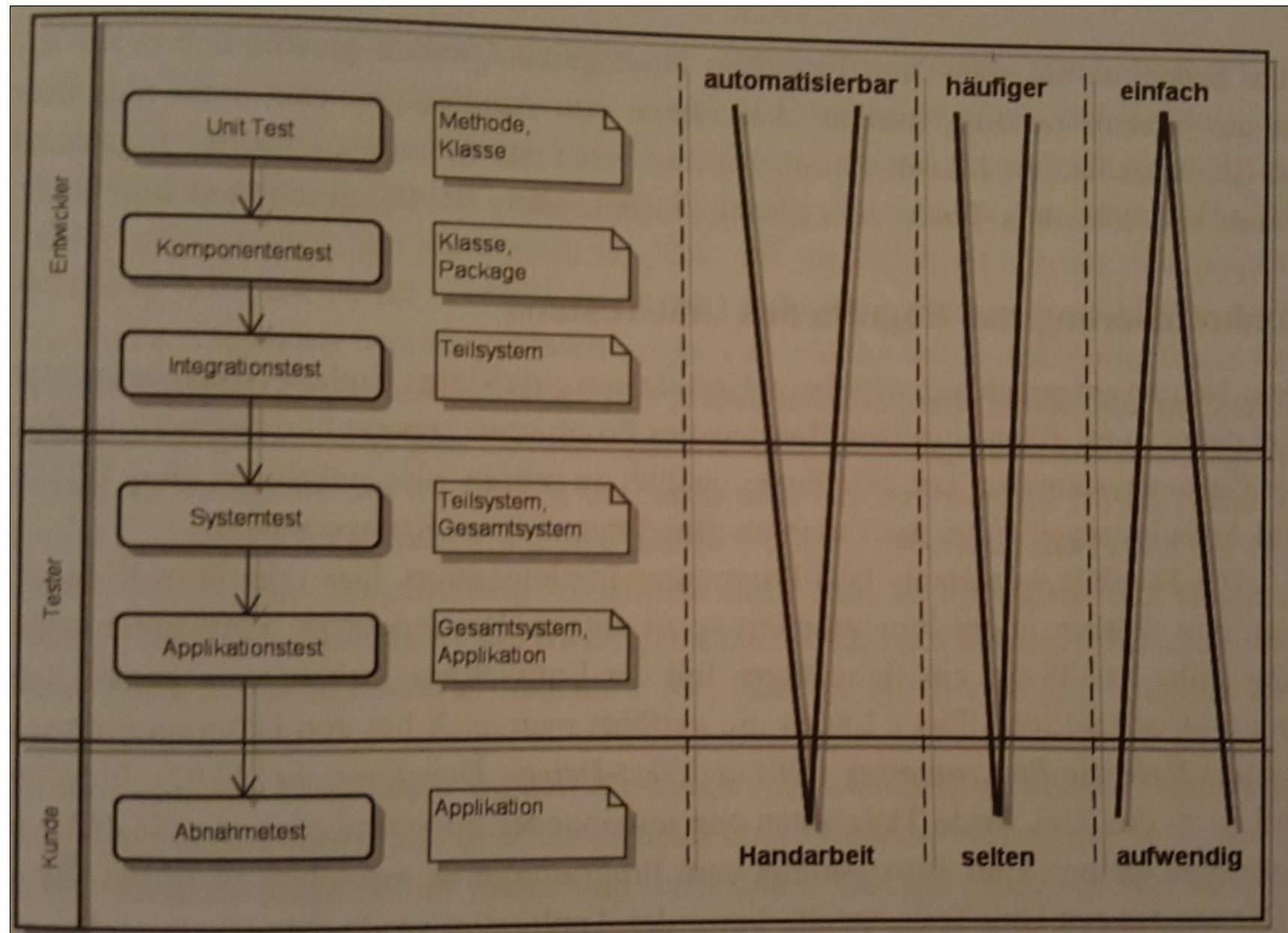
Testebenen

- Manuelles Testen (Quelltextzeilen, Methoden)
- Modultest, Unit-Test
 - Methode, Klasse, Funktionale Einheiten
- Komponententest (Klasse, Paket)
- Integrationstest (Teilsystem)
- Systemtest (Teilsystem, Gesamtsystem)
- Applikationstest (Gesamtsystem, Applikation)
- Abnahmetest (Applikation)

Zeit



Testebenen



Manuelles Testen während Implementierung

- **Frühe Prüfungen** zur Qualitätssteigerung sind sinnvoll und manuell schon vor der Durchführung von Modultest möglich:
 - Debuggen (Durchsteppen mit Debugger)
 - Methoden aufrufen und manuell Ergebnisse prüfen
 - Zwischenergebnisse auf der Konsole ausgeben ("printf")
 - Code-Inspektion = Quelltext in Gedanken durchgehen
 - Exploratives Testen (Software "erkunden")
 - usw.

Modultest = Unit-Test

- Frühe Teststufe
- Kann gut vom Entwickler erstellt werden
- Unterstützung durch Software und Automatisierung (Regressionstests) mittels Build-Server
 - Tools z.B. für Java: JUnit, Hamcrest, Cobertura, EclEmma, Infinitest, ...
- Je Testfall eigene Testmethode schreiben
- Alle Testmethoden in Testklasse bündeln



Modultest = Unit-Test

- Ablauf je Testfall (Testmethode):
 1. Ausgangszustand herstellen
 2. Zu testende Operation(en) ausführen
 3. Ergebnis dieser Operation(en) mit Wunscheregebnis abgleichen
 - Mittels *assert*-Anweisungen
 - `assert addition(2,3) == 5;`
 - `assert argc == 4;`
 - ...

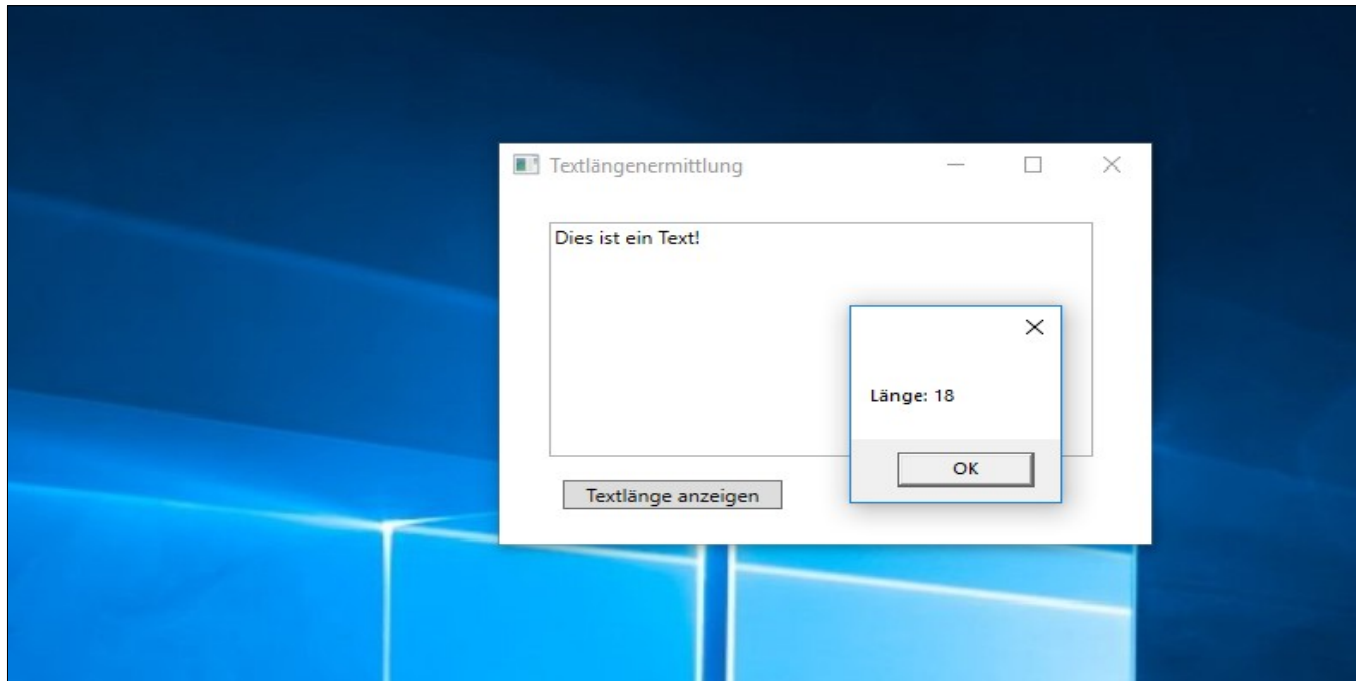
Der fundamentale Testfall

- Grundlegend hat jeder Testfall mindestens $2 \times 2 = 4$ essentielle Bestandteile:
- Definition:
 - Testschritte "Tue A, B, C, D"
 - Erwartetes Ergebnis "Es soll XYZ passieren"
- Auswertung:
 - Tatsächliches Ergebnis "XYZ passiert"
 - Resultat "OK"

Der fundamentale Testfall

- Schritte:
 - Programm starten
 - Text "Dies ist ein Text!" eingeben
 - Button drücken
- Erwartetes Ergebnis:

Meldung mit "Länge: 18" erscheint



Erweiterter Testfall

Testfall	
Nummer	Die Nummer des Testfalls
Titel	Name des Testfalls
Software	Mit welcher Software und Software-Version der Testfall durchzuführen ist
Beschreibung	Allgemeine Beschreibung, was hier wie getestet wird
Vorbedingungen	Eventuelle Vorbedingungen, die erst vorhanden sein müssen, um den Testfall durchzuführen
Schritte	Welche Schritte sind exakt notwendig um diesen Testfall durchzuführen
Erwartetes Ergebnis	Welches Ergebnis/Verhalten von der Software erwartet wird
Anhänge	Zum Beispiel Screenshots oder Diagramme, die den Testfall näher erläutern
Priorität	In welcher Reihenfolge (in Bezug auf andere Testfälle) dieser Testfall durchgeführt werden soll (z.B. Nummer von 1 bis 99)
Kommentar	Zusätzliche Anmerkungen

Logische und konkrete Testfälle

Beispiel: Das Schwimmbad möchte Ticketpreise je nach Alter vergeben, und zwar Rabatte von 30% für unter 18-jährige und Rabatte von 40% für über 63-jährige.

Logische Testfälle:

Testfall	1	2	3
Eingabe x (Alter)	$x < 18$	$18 \leq x \leq 63$	$x > 63$
Erwartetes Ergebnis (Rabatt in Prozent)	30	0	40

Konkrete Testfälle:

Testfall	1	2	3
Eingabe x (Alter)	12	40	70
Erwartetes Ergebnis (Rabatt in Prozent)	30	0	40

Der fundamentale Testprozess

- ◆ Der fundamentale Testprozess gliedert sich in 5 Schritte:

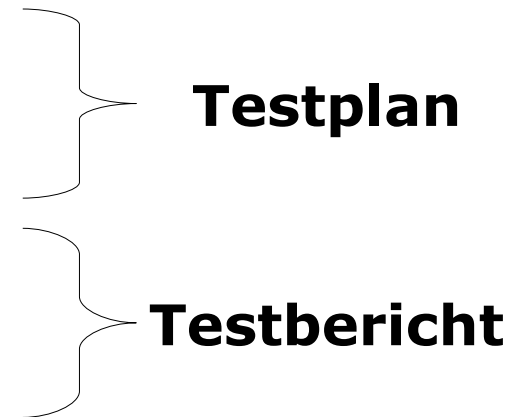
1. Testplanung und -steuerung

2. Testanalyse und -entwurf

3. Testrealisierung und -durchführung

4. Testauswertung und -bericht

5. Testabschluss



- ◆ Dokumente:

- **Testplan** (1.+2.)

- **Testbericht** (3.+4.)

Der fundamentale Testprozess

1. Testplanung und -steuerung

- Teststrategie, Risikoeinschätzung der Funktionen, Testende-Kriterien, Definition Fortschrittsüberwachung, Ressourcen einplanen

2. Testanalyse und -entwurf

- Sind Informationen vollständig zur Erstellung von Testfällen?
 - z.B. Pflichtenheft, Anforderungen, Abnahmekriterien, ...
- Testspezifikation in **Testplan** eintragen:
 - Liste von Testfällen erstellen
 - Erst Logisch, dann Konkret

Beispiel für Teststrategie

"Die Software wird insbesondere von Ingenieuren im technischen Umfeld angewendet. Die Formeln für die technischen Berechnungen müssen unbedingt korrekt funktionieren. Die Oberflächen sollten insbesondere dahingehend getestet werden, dass keine gravierenden Fehleingaben möglich sind."

Der fundamentale Testprozess

3. Testrealisierung und -durchführung

- Aufbau der Testumgebung
(Hardware, Software installieren, System einrichten, ...)
- Abarbeiten der Testfälle
- Protokollieren der Ergebnisse
 - Gefundene Fehler (*Defects*) auflisten

4. Testauswertung und -bericht

- Auswertung der Testergebnisse (Resultate)
- Prüfen auf Testende: Sind alle Anforderungen getestet (oder Zeit oder Kosten ausgeschöpft)?
- **Testbericht** erstellen mit Kurzberichten für Stakeholdern sowie Freigabeempfehlung

Regressionstests

- Falls eine neue Software-Version fertiggestellt ist, sollten alle Tests wiederholt werden
 - = "Regressionstests"
 - Dies stellt sicher, dass bei Änderungen keine neuen Fehler eingebaut wurden
 - Praktisch ist dies generell nur durch Automatisierung mit Hilfe von Tools umsetzbar

Der fundamentale Testprozess

5. Testabschluss

- Alle *Defect*-Tickets schließen
 - Status entweder auf "Erledigt" oder "Behebung nicht (für diese Version) geplant" setzen
 - Falls eine neue Software-Version fertiggestellt werden soll, sollten nun alle noch offenen und ungelösten Defect-Tickets abgeschlossen werden
- Archivierung der Testumgebung für eventuelles Nachvollziehen zu einem späteren Zeitpunkt
- Reflektion der Testaktivitäten: Was war positiv, was war negativ, was könnte man verbessern?

Inhalte des Testplans

- Allgemeine Informationen (Projekt, Release, Ersteller, Datum, ...)
- Teststrategie (worauf wird der Fokus gelegt)
- Testablauf (wie wird vorgegangen)
- Bezug auf Anforderungen (Lastenheft, Pflichtenheft, ...)
- Liste der Logischen und Konkreten Testfälle
- Zeitplanung für die Durchführung des Testplans
- Testaufzeichnungsprozedur (wie werden Testergebnisse dokumentiert)
- Anforderungen (an Hardware, Software, Umgebung, ...)
- Einschränkungen (Auswirkungen durch Personalmangel, kein Netzwerkzugriff, Hardwareverfügbarkeit, ...)

Inhalte des Testberichts

- Allgemeine Informationen (Projekt, Release, Bearbeiter, Datum, ...)
- Informationen zu genutzten Hilfsmitteln (Hardware, Software, System, Umgebung, Vorbedingungen, Schnittstellen, ...)
- Testergebnisse je Testfall gemäß Testaufzeichnungsprozedur
 - z.B. Anlegen einer extra *Defect*-Liste
- Fazit (sind alle Anforderungen getestet, Detailhinweise auf mögliche Fehlerquellen, wo und welche Fehler gab es, Empfehlungen für weitere Tests, Gesamtbewertung, ...)
- Verschiedene, spezifische Kurzberichte für diverse Stakeholder (Test-Manager, Entwickler, Projektleiter, ...)
- Empfehlung zur Freigabe

Weitere Werkzeuge zur Qualitätssicherung

- Einsatz von Vorgehensmodellen generell
- Richtlinien für Programmierung, Dokumentation
- Formatvorlagen
- Traceability von Anforderungen nutzen
- Einsatz von Tools:
 - Versionsmanagement, Regressionstests, Build-Servern ("Nightly Builds"), Debugger, Static-Code-Analysis, ...
- Ethische Regeln
- usw.

Tools für Testmanagement und Build-Server

- **Kommerziell:**

- Jenkins <https://jenkins.io/>
- Atlassian Bamboo
<https://www.atlassian.com/software/bamboo>

- **Frei verfügbar:**

- BuildBot <https://buildbot.net/>
- Travis CI <https://travis-ci.org/>
- JetBrains TeamCity
<https://www.jetbrains.com/teamcity/>

- u.A.

Zusammenfassung

- Das Test and Quality Management (TQM) verwaltet Testpläne, das Testen sowie Maßnahmen zur Qualitätsverbesserung der Software
 - Fundamentaler Bestandteil ist **Testen**
 - Unterstützt durch Testpläne, Testberichte, Testfälle, usw.
 - In verschiedenen **Phasen**
 - Auf verschiedenen **Ebenen**
 - Die Verifizierung und Validierung (V&V) der Software kann **statisch** und **dynamisch** ablaufen
 - Tools unterstützen dabei und können den Testprozess automatisieren

Fragen?